Dylan Thornburg
ECEN 603

HW1

For each of the following problems, submit the Verilog code for both the design modules and the test bench. Additionally, submit the simulation output.

1. Given the following dataflow Verilog code for a 1-bit CSA module, design an 8-bit Carry-save adder using structural Verilog.

```
module CSA1bit(
        input a,b,c,
        output sum,carry
);
        assign sum = a ^ b ^ c;
        assign carry = (a & b) | (c & b) | (a & c);
endmodule
```

8-bit CSA Code

```
module CSA8bit(
        input [7:0] a,b,c,
        output [7:0] sum, carry
);
        CSA1bit zero(
                .a(a[0]),
                .b(b[0]),
                .c(c[0]),
                .sum(sum[0]),
                .carry(carry[0])
        );

        CSA1bit one(
                .a(a[1]),
                .b(b[1]),
                .c(c[1]),
                .sum(sum[1]),
                .carry(carry[1])
        );

        CSA1bit two(
                .a(a[2]),
                .b(b[2]),
                .c(c[2]),
                .sum(sum[2]),
                .carry(carry[2])
        );

        CSA1bit three(
                .a(a[3]),
```

```
                .b(b[3]),
                .c(c[3]),
                .sum(sum[3]),
                .carry(carry[3])
        );

        CSA1bit four(
                .a(a[4]),
                .b(b[4]),
                .c(c[4]),
                .sum(sum[4]),
                .carry(carry[4])
        );

        CSA1bit five(
                .a(a[5]),
                .b(b[5]),
                .c(c[5]),
                .sum(sum[5]),
                .carry(carry[5])
        );

        CSA1bit six(
                .a(a[6]),
                .b(b[6]),
                .c(c[6]),
                .sum(sum[6]),
                .carry(carry[6])
        );

        CSA1bit seven(
                .a(a[7]),
                .b(b[7]),
                .c(c[7]),
                .sum(sum[7]),
                .carry(carry[7])
        );

endmodule
```

8-bit CSA TB Code

```
module testbench();
        reg [7:0] a,b,c;
        wire [7:0] sum, carry;
        CSA8bit(a,b,c,sum,carry);
        initial begin
                $display("Start of the 8-bit CSA test. Expect a+b+c = sum + (carry << 1)");
                $monitor("a=%d,b=%d,c=%d,sum=%d,carry=%d,carry << 1=%d, @ %d",
a,b,c,sum,carry,(carry<<1),$time);
                a=8'b0; b= 8'b0; c = 8'b0;
                #10;
                a=8'b10000000; b= 8'b01110000; c = 8'b0;
```

```
                #10;
                a=8'b00110001; b= 8'b01100110; c = 8'b00001111;
                #10;
                a=8'b10101010; b= 8'b01010101; c = 8'b00000001;
                #10;
                $display("End of the test for 8-bit CSA");
                $finish;
        end
endmodule
```

TEST RESULTS

```
[dthornburg@linux402604 HW1]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;
  Jan 29 15:35 2026
Start of the 8-bit CSA test. Expect a+b+c = sum + (carry << 1)
a=  0,b=  0,c=  0,sum=  0,carry=  0,carry << 1=  0, @                    0
a=128,b=112,c=  0,sum=240,carry=  0,carry << 1=  0, @                   10
a= 49,b=102,c= 15,sum= 88,carry= 39,carry << 1= 78, @                   20
a=170,b= 85,c=  1,sum=254,carry=  1,carry << 1=  2, @                   30
End of the test for 8-bit CSA
$finish called from file "CSA8bit_tb.v", line 17.
$finish at simulation time                   40
           V C S   S i m u l a t i o n   R e p o r t
Time: 40
CPU Time:      0.170 seconds;       Data structure size:   0.0Mb
Thu Jan 29 15:35:44 2026
[dthornburg@linux402604 HW1]$
```

Results match what's expected. Since we dont have the CPA, we expect a+b+c=sum+ 2*carry
(carry <<1)
128+112+0=240+0
49+102+15=88+2*39=166
170+85+1=254+2*1=256

2. Using Verilog code, implement the following circuit (Fig. 7.16 of the text) with 8-bit Carry-save adders that can add seven 8-bit numbers as shown below:

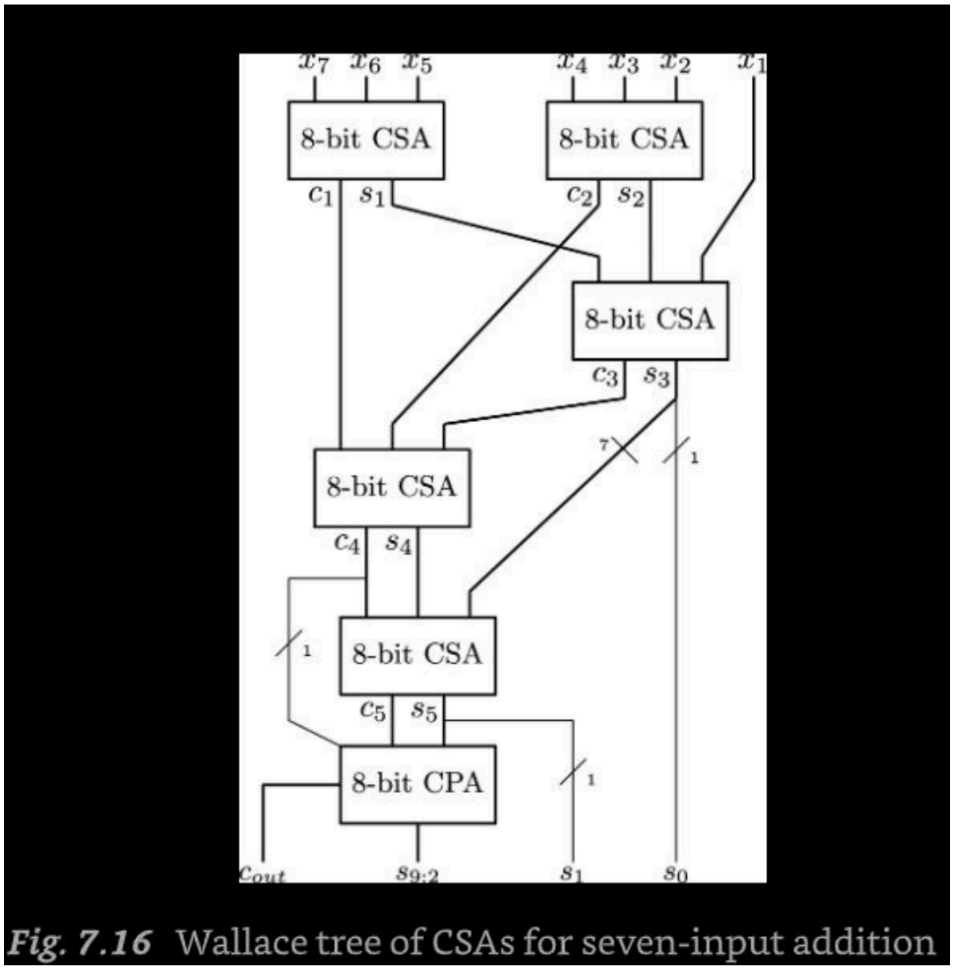

Fig. 7.16 Wallace tree of CSAs for seven-input addition

(a) Use a carry-propagate adder in the last step

8-BIT CPA CODE

```
module CPA8bit(
        input [7:0] a,b,
        input cin,
        output [7:0] sum,
        output cout
);
        wire [8:0] sum_temp;
        assign sum_temp = a+b+cin;
        assign sum = sum_temp[7:0];
        assign cout = sum_temp[8];
endmodule
```

WALLACE TREE CODE

```
module WallaceTree7_8bit(
        input [7:0] x1,
        input [7:0] x2,
        input [7:0] x3,
```

```
        input [7:0] x4,
        input [7:0] x5,
        input [7:0] x6,
        input [7:0] x7,
        output [9:0] sum,
        output cout
);
        wire [7:0] s1;
        wire [7:0] c1;
        CSA8bit one(
                .a(x7),
                .b(x6),
                .c(x5),
                .sum(s1),
                .carry(c1)
        );

        wire [7:0] s2;
        wire [7:0] c2;
        CSA8bit two(
                .a(x4),
                .b(x3),
                .c(x2),
                .sum(s2),
                .carry(c2)
        );

        wire [7:0] s3;
        wire [7:0] c3;
        CSA8bit three(
                .a(s1),
                .b(s2),
                .c(x1),
                .sum(s3),
                .carry(c3)
        );

        wire [7:0] s4;
        wire [7:0] c4;
        CSA8bit four(
                .a(c1),
                .b(c2),
                .c(c3),
                .sum(s4),
                .carry(c4)
        );

        wire [7:0] s5;
        wire [7:0] c5;
        CSA8bit five(
                .a({c4[6:0],1'b0}),
                .b(s4),
                .c({1'b0,s3[7:1]}),
                .sum(s5),
                .carry(c5)
```

```
        );

        wire [7:0] s6;
        wire cout_cpa;
        CPA8bit cpa_one(
                .a(c5),
                .b({c4[7],s5[7:1]}),
                .cin(0),
                .sum(s6),
                .cout(cout_cpa)
        );

        assign cout = cout_cpa;
        assign sum = {s6,s5[0],s3[0]};

endmodule
```

TEST RESULTS

```
[dthornburg@linux402604 HW1]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Jan 30 00:47 2026
Start of the Wallace Tree Test.
x1=  0,x2=  0,x3=  0,x4=  0,x5=  0,x6=  0,x7=  0,sum=   0,cout=0, @                    0
x1=  1,x2=  2,x3=  3,x4=  4,x5=  5,x6=  6,x7=  7,sum=  28,cout=0, @                   10
x1= 12,x2= 34,x3= 56,x4= 78,x5= 90,x6= 45,x7=  8,sum= 323,cout=0, @                   20
x1=255,x2=255,x3=255,x4=255,x5=255,x6=255,x7=255,sum= 761,cout=1, @                   30
End of the Wallace Tree test
$finish called from file "WallaceTree7_8bit_tb.v", line 20.
$finish at simulation time                   40
           V C S   S i m u l a t i o n   R e p o r t
Time: 40
CPU Time:      0.170 seconds;       Data structure size:   0.0Mb
Fri Jan 30 00:47:40 2026
[dthornburg@linux402604 HW1]$
```

All is expected. 1024 (cout) + 761 = 1785 = 255*7

(b) Replace the carry-propagate adder with a carry lookahead adder in the last step

CODE CHANGES

```
// ALL THE SAME ABOVE
wire [7:0] s6;
        wire cout_cla;
        //wire cout_cpa;
        //CPA
        /*
        CPA8bit cpa_one(
                .a(c5),
                .b({c4[7],s5[7:1]}),
                .cin(0),
                .sum(s6),
```

```
                .cout(cout_cpa)
        );
        */
        CLA8bit cla_one(
                .a(c5),
                .b({c4[7],s5[7:1]}),
                .cin(0),
                .sum(s6),
                .cout(cout_cla)
        );
        assign cout = cout_cla;
//ALL THE SAME BELOW
```

```
module CLA8bit(
        input [7:0] a,b,
        input cin,
        output [7:0] sum,
        output cout
);
        wire [7:0] g,p,c;
        assign p = a ^ b;
        assign g = a & b;

        assign c[0] = g[0] | (p[0] & cin);

        assign c[1] = g[1] | (p[1] & g[0]) |
                        (p[1] & p[0] & cin);

        assign c[2] = g[2] | (p[2] & g[1]) |
                    (p[2] & p[1] & g[0]) |
                    (p[2] & p[1] & p[0] & cin);

        assign c[3] = g[3] | (p[3] & g[2]) |
                        (p[3] & p[2] & g[1]) |
                        (p[3] & p[2] & p[1] & g[0]) |
                        (p[3] & p[2] & p[1] & p[0] & cin);

        assign c[4] = g[4] | (p[4] & g[3]) |
                        (p[4] & p[3] & g[2]) |
                        (p[4] & p[3] & p[2] & g[1]) |
                        (p[4] & p[3] & p[2] & p[1] & g[0]) |
                        (p[4] & p[3] & p[2] & p[1] & p[0] & cin);

        assign c[5] = g[5] | (p[5] & g[4]) |
                        (p[5] & p[4] & g[3]) |
                        (p[5] & p[4] & p[3] & g[2]) |
                        (p[5] & p[4] & p[3] & p[2] & g[1]) |
                        (p[5] & p[4] & p[3] & p[2] & p[1] & g[0]) |
                        (p[5] & p[4] & p[3] & p[2] & p[1] & p[0] & cin);

        assign c[6] = g[6] | (p[6] & g[5]) |
                        (p[6] & p[5] & g[4]) |
                        (p[6] & p[5] & p[4] & g[3]) |
```

```
                        (p[6] & p[5] & p[4] & p[3] & g[2]) |
                        (p[6] & p[5] & p[4] & p[3] & p[2] & g[1]) |
                        (p[6] & p[5] & p[4] & p[3] & p[2] & p[1] & g[0]) |
                        (p[6] & p[5] & p[4] & p[3] & p[2] & p[1] & p[0] & cin);

        assign c[7] = g[7] | (p[7] & g[6]) |
                        (p[7] & p[6] & g[5]) |
                        (p[7] & p[6] & p[5] & g[4]) |
                        (p[7] & p[6] & p[5] & p[4] & g[3]) |
                        (p[7] & p[6] & p[5] & p[4] & p[3] & g[2]) |
                        (p[7] & p[6] & p[5] & p[4] & p[3] & p[2] & g[1]) |
                        (p[7] & p[6] & p[5] & p[4] & p[3] & p[2] & p[1] & g[0]) |
                        (p[7] & p[6] & p[5] & p[4] & p[3] & p[2] & p[1] & p[0] & cin);

        assign sum[0] = p[0] ^ cin;
        assign sum[1] = p[1] ^ c[0];
        assign sum[2] = p[2] ^ c[1];
        assign sum[3] = p[3] ^ c[2];
        assign sum[4] = p[4] ^ c[3];
        assign sum[5] = p[5] ^ c[4];
        assign sum[6] = p[6] ^ c[5];
        assign sum[7] = p[7] ^ c[6];

        assign cout = c[7];

endmodule
```

TEST RESULTS

```
[dthornburg@linux402604 HW1]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Jan 30 01:18 2026
Start of the Wallace Tree Test.
x1=  0,x2=  0,x3=  0,x4=  0,x5=  0,x6=  0,x7=  0,sum=   0,cout=0, @                    0
x1=  1,x2=  2,x3=  3,x4=  4,x5=  5,x6=  6,x7=  7,sum=  28,cout=0, @                   10
x1= 12,x2= 34,x3= 56,x4= 78,x5= 90,x6= 45,x7=  8,sum= 323,cout=0, @                   20
x1=255,x2=255,x3=255,x4=255,x5=255,x6=255,x7=255,sum= 761,cout=1, @                   30
End of the Wallace Tree test
$finish called from file "WallaceTree7_8bit_tb.v", line 20.
$finish at simulation time                   40
           V C S   S i m u l a t i o n   R e p o r t
Time: 40
CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
Fri Jan 30 01:18:15 2026
[dthornburg@linux402604 HW1]$
```

These results match the previous ones.